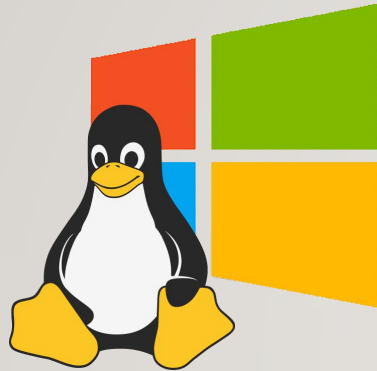




SO, PROCESSOS E ESCALONAMENTO

SERVIDORES E SEUS SISTEMAS OPERACIONAIS

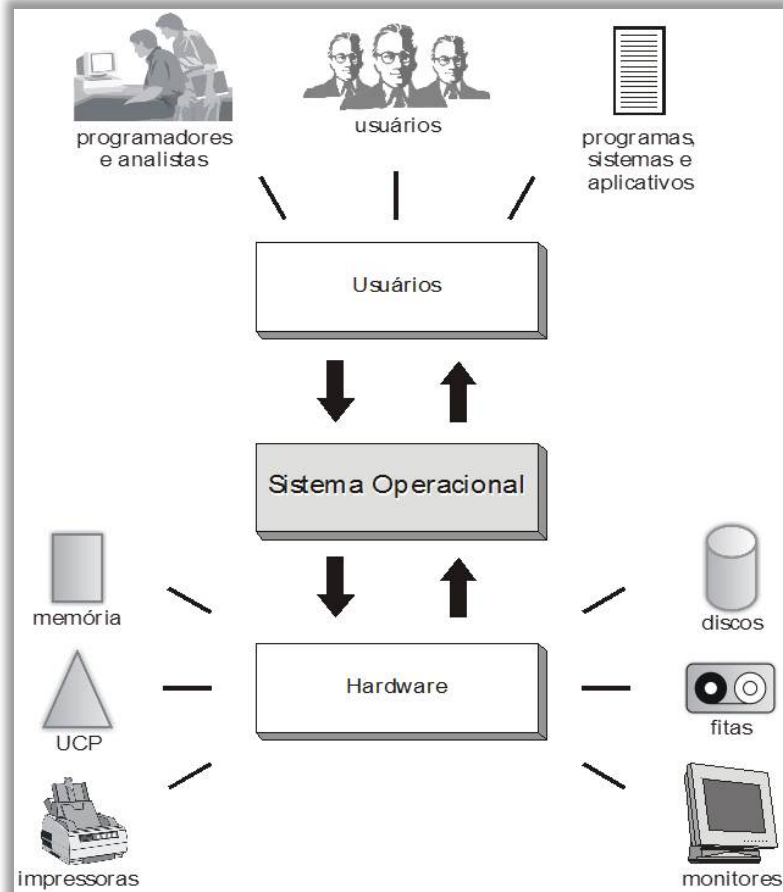
Prof.: Mauricio de Oliveira Dian

E-mail: mauricio.dian@fatectq.edu.br

NA AULA PASSADA...

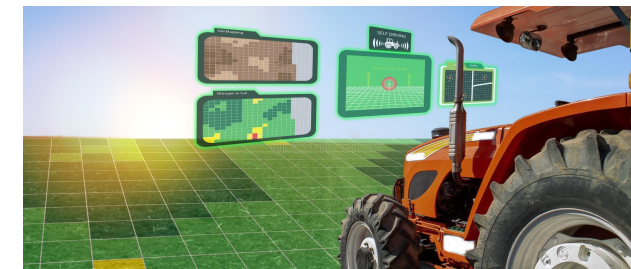
- Conceitos sobre o que são Sistemas Operacionais
- Histórico e Evolução dos SOs
- Tipos e aplicações de Sistemas Operacionais
- Arquitetura de um Sistema Computacional
- Arquitetura e Estrutura dos Sistemas Operacionais

NA AULA PASSADA...



Visão do Sistema Operacional

Fonte: *Arquitetura de Sistemas Operacionais*, 4ª Edição, 2007



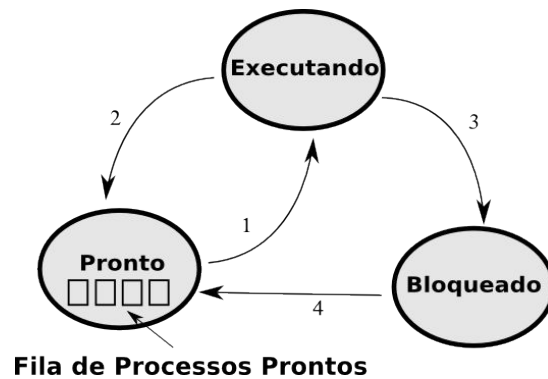
Sistemas Operacionais em diversos dispositivos

Fonte: Google Imagens

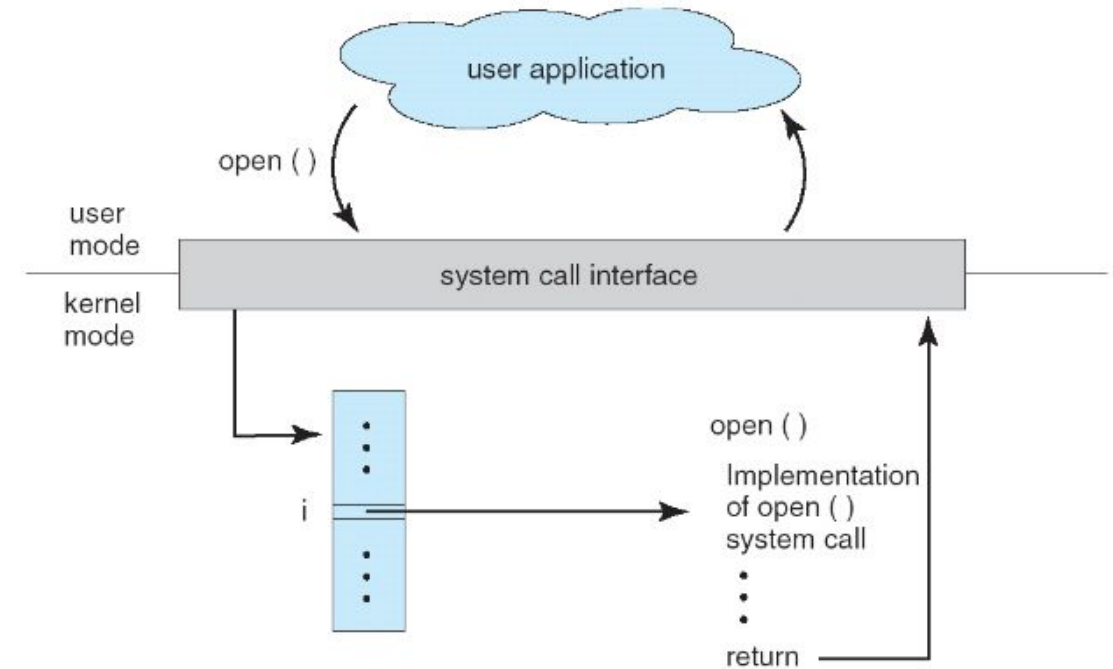
NA AULA PASSADA...



Arquitetura de um Sistema Computacional
Fonte: Google Imagens



Estados de um Processo
Fonte: Google Imagens



Chamada de Sistema `open()`
Fonte: Google Imagens

OBJETIVOS

- Conceitos sobre o que são processos, mudanças e seus estados
- Processos *Foreground* e *Background*, *CPU-bound* e *I/O-bound*
- Conceitos sobre *Threads*, *Monothreads* e *Multithreads*
- Escalonador e Algoritmos de Escalonamento
- Explorar o Simulador SOsim



Fonte: Google Imagens



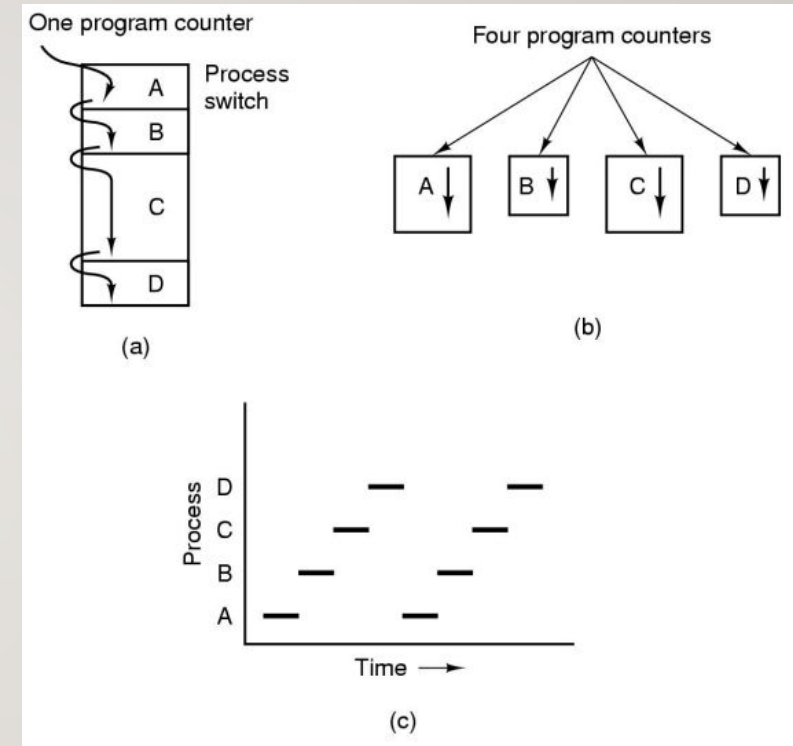
Vamos pensar um pouco ?

Pense por alguns segundos...

1. Como são os sistemas operacionais e servidores atuais (Monoprogramáveis ou Multiprogramáveis)?
2. Como lidam com as várias tarefas que queremos fazer no nosso dia a dia?

PROCESSOS

- Programas em execução que possuem dados de entrada, dados de saída e um estado.
- Os usuários e o sistema iniciam e finalizam processos a todo instante.
- **Multiprogramação** □ **Pseudoparalelismo:**
 - Coleção de processos sendo executados alternativamente na CPU.



- (a) Modelo de multiprogramação com 4 processos na memória.
(b) Modelo conceitual de 4 processos sequenciais e independentes.
(c) Somente um programa está ativo de cada vez.

Fonte: Google Imagens

PROCESSOS

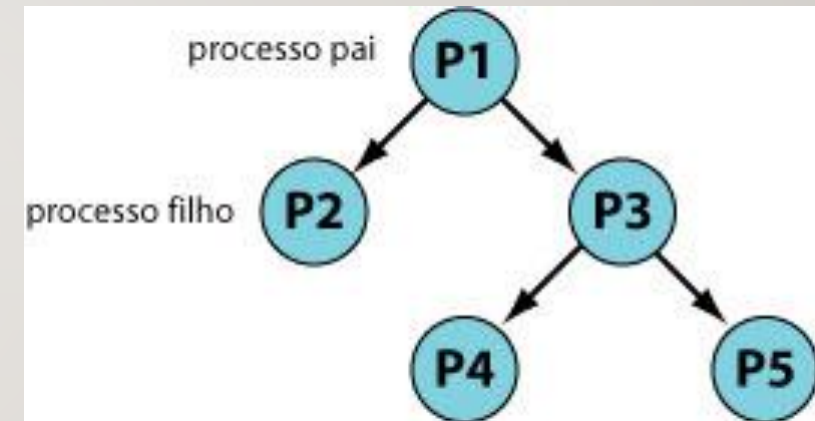
- **Processos Independentes:**

- Processos sem vínculos com outros processos.

- **Subprocessos (processos-filhos):**

- São processos que possuem um vínculo existencial com outro processo (processo-pai).
- Se o processo-pai é finalizado, toda sua hierarquia de filhos também deixa de existir.

- Nos Linux, por exemplo: o “pai de todos” é o “init” (systemd).

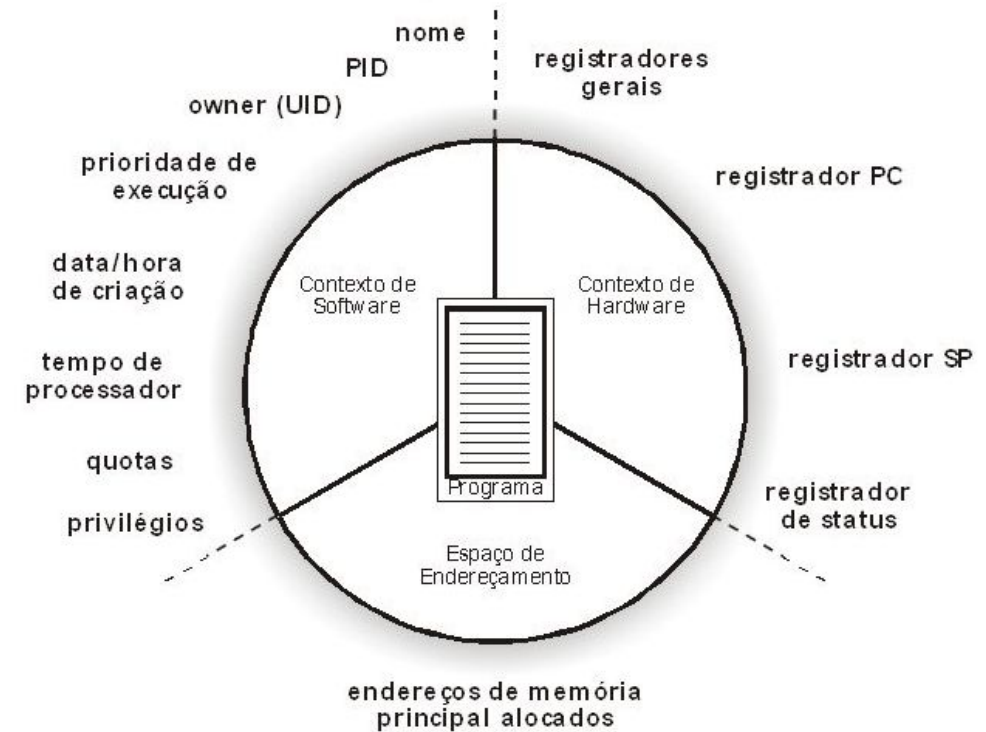


Fonte: Google Imagens

ESTRUTURA DE UM PROCESSO DETALHADO

- **Processos são compostos por:**

- ❑ **Contexto de Hardware:** conteúdo dos registradores gerais e de uso específico usado para retomar a execução do processo onde parou.
- ❑ **Contexto de Software:** limites e características dos recursos que podem ser alocados pelo processo.
- ❑ **Espaço de endereçamento:** área da memória principal usada pelo processo, onde as instruções e dados do programa são armazenados para execução.
- ❑ **Programa:** sequência de instruções, composta por desvios, repetições e chamadas a funções e procedimentos.



Características da Estrutura de um Processo

Fonte: Arquitetura de Sistemas Operacionais, 4ª Edição, 2007

PROCESSOS

- **Os 5 estados de um processo:**

- **Novo:** quando a tarefa está sendo criada (carregada na memória).
- **Pronto:** quando a tarefa está em memória aguardando a disponibilidade do processador para ser executada.
- **Executando:** quando o processador está executando a tarefa em questão.
- **Suspenso:** quando a tarefa está a espera de um evento ou dado externo para estar pronta novamente para execução.
- **Terminado:** quando a tarefa em questão foi totalmente finalizada e já pode sair da memória.

PROCESSOS

- **No Linux:**

- ❑ **R** : Execução
- ❑ **S** : Suspenso
- ❑ **T** : Interrompido
- ❑ **Z** : Zumbi

Comando: `ps -aux`

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.5	164356	10756	?	Ss	17:24	0:05	/sbin/init
root	2	0.0	0.0	0	0	?	S	17:24	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	I<	17:24	0:00	[rcu_gp]
root	4	0.0	0.0	0	0	?	I<	17:24	0:00	[rcu_par_gp]
root	6	0.0	0.0	0	0	?	I<	17:24	0:00	[kworker/0:0H-events_highpri]
root	8	0.0	0.0	0	0	?	I<	17:24	0:00	[mm_percpu_wq]
root	9	0.0	0.0	0	0	?	S	17:24	0:00	[rcu_tasks_rude_]
root	10	0.0	0.0	0	0	?	S	17:24	0:00	[rcu_tasks_trace]
root	11	0.0	0.0	0	0	?	S	17:24	0:00	[ksoftirqd/0]
root	12	0.0	0.0	0	0	?	I	17:24	0:01	[rcu_sched]
root	13	0.0	0.0	0	0	?	S	17:24	0:00	[migration/0]
root	14	0.0	0.0	0	0	?	S	17:24	0:00	[idle_inject/0]
root	16	0.0	0.0	0	0	?	S	17:24	0:00	[cpuhp/0]
root	17	0.0	0.0	0	0	?	S	17:24	0:00	[cpuhp/1]
root	18	0.0	0.0	0	0	?	S	17:24	0:00	[idle_inject/1]
root	19	0.0	0.0	0	0	?	S	17:24	0:00	[migration/1]
root	20	0.0	0.0	0	0	?	S	17:24	0:00	[ksoftirqd/1]
root	22	0.0	0.0	0	0	?	I<	17:24	0:00	[kworker/1:0H-kblockd]
root	23	0.0	0.0	0	0	?	S	17:24	0:00	[kdevtmpfs]
root	24	0.0	0.0	0	0	?	I<	17:24	0:00	[netns]
root	25	0.0	0.0	0	0	?	I<	17:24	0:00	[inet_frag_wq]
root	26	0.0	0.0	0	0	?	S	17:24	0:00	[kauditd]
root	27	0.0	0.0	0	0	?	S	17:24	0:00	[khungtaskd]
root	28	0.0	0.0	0	0	?	S	17:24	0:00	[oom_reaper]
root	29	0.0	0.0	0	0	?	I<	17:24	0:00	[writeback]
root	30	0.0	0.0	0	0	?	S	17:24	0:00	[kcompactd0]

Fonte: Próprio autor

DIAGRAMA DE ESTADOS DE UM PROCESSO

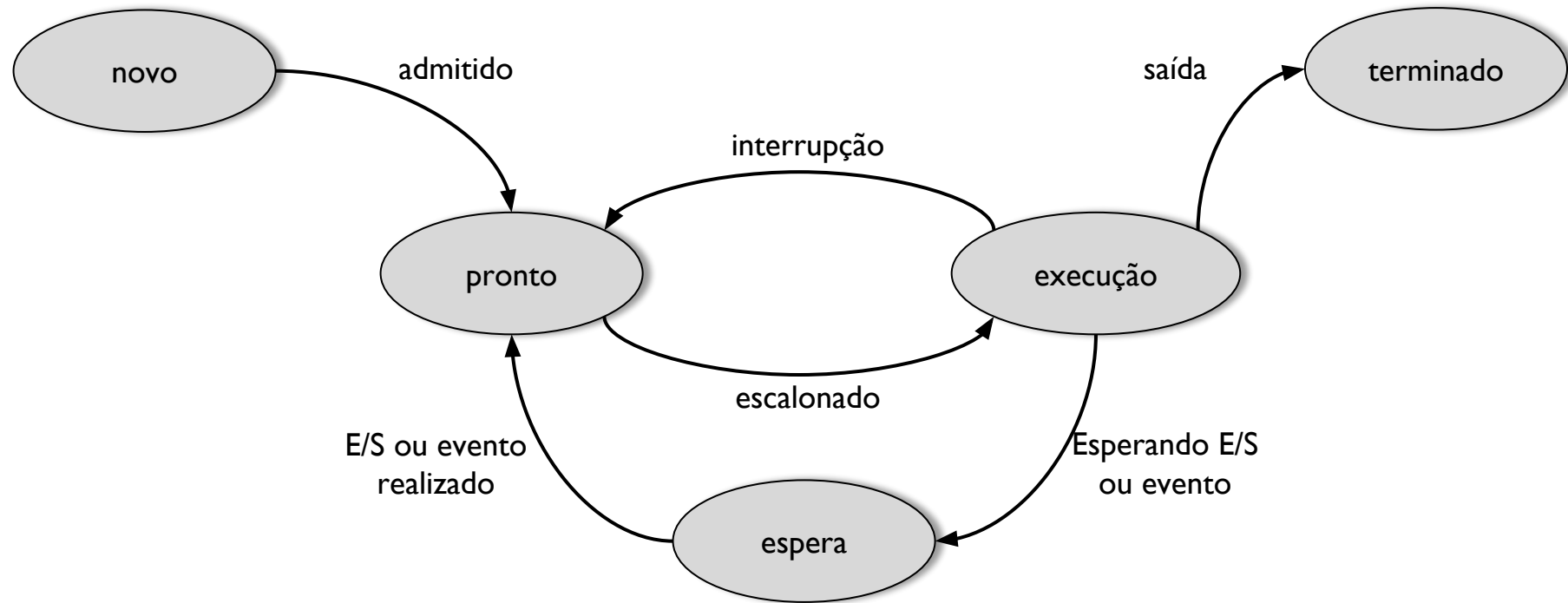


Diagrama de Estado dos Processos

Fonte: Google Imagens

Comando: ps -aux

```
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.5 164356 10756 ?        Ss   17:24   0:05 /sbin/init
root         2  0.0  0.0      0     0 ?        S    17:24   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        I<   17:24   0:00 [rcu_gp]
root         4  0.0  0.0      0     0 ?        I<   17:24   0:00 [rcu_par_gp]
root         6  0.0  0.0      0     0 ?        I<   17:24   0:00 [kworker/0:0H-events_highpri]
root         8  0.0  0.0      0     0 ?        I<   17:24   0:00 [mm_percpu_wq]
root         9  0.0  0.0      0     0 ?        S    17:24   0:00 [rcu_tasks_rude_]
root        10  0.0  0.0      0     0 ?        S    17:24   0:00 [rcu_tasks_trace]
root        11  0.0  0.0      0     0 ?        S    17:24   0:00 [ksoftirqd/0]
root        12  0.0  0.0      0     0 ?        I    17:24   0:01 [rcu_sched]
root        13  0.0  0.0      0     0 ?        S    17:24   0:00 [migration/0]
root        14  0.0  0.0      0     0 ?        S    17:24   0:00 [idle_inject/0]
root        16  0.0  0.0      0     0 ?        S    17:24   0:00 [cpuhp/0]
root        17  0.0  0.0      0     0 ?        S    17:24   0:00 [cpuhp/1]
root        18  0.0  0.0      0     0 ?        S    17:24   0:00 [idle_inject/1]
root        19  0.0  0.0      0     0 ?        S    17:24   0:00 [migration/1]
root        20  0.0  0.0      0     0 ?        S    17:24   0:00 [ksoftirqd/1]
root        22  0.0  0.0      0     0 ?        I<   17:24   0:00 [kworker/1:0H-kblockd]
root        23  0.0  0.0      0     0 ?        S    17:24   0:00 [kdevtmpfs]
root        24  0.0  0.0      0     0 ?        I<   17:24   0:00 [netns]
root        25  0.0  0.0      0     0 ?        I<   17:24   0:00 [inet_frag_wq]
root        26  0.0  0.0      0     0 ?        S    17:24   0:00 [kauditd]
root        27  0.0  0.0      0     0 ?        S    17:24   0:00 [khungtaskd]
root        28  0.0  0.0      0     0 ?        S    17:24   0:00 [oom_reaper]
root        29  0.0  0.0      0     0 ?        I<   17:24   0:00 [writeback]
root        30  0.0  0.0      0     0 ?        S    17:24   0:00 [kcompactd0]
```

Comando: ps -f -u jose

```
ubuntu@ubuntu-server:~$
ubuntu@ubuntu-server:~$
ubuntu@ubuntu-server:~$ ps -f -u jose
UID      PID PPID  C STIME TTY      TIME CMD
jose     2153 1    0 18:59 ?        00:00:00 /lib/systemd/systemd --user
jose     2154 2153 0 18:59 ?        00:00:00 (sd-pam)
jose     2159 2084 0 18:59 tty1    00:00:00 -bash
jose     2315 2159 0 19:47 tty1    00:00:01 ping www.google.com
ubuntu@ubuntu-server:~$
ubuntu@ubuntu-server:~$
```

Comando: pstree -p

```
systemd(1) --accounts-daemon(627) --{accounts-daemon}(654)
    |--{accounts-daemon}(690)
    |--agetty(1445)
    |--cron(629)
    |--dbus-daemon(630)
    |--irqbalance(634) ---{irqbalance}(638)
    |--login(2084) ---bash(2159) ---ping(2315)
    |--login(2208) ---bash(2284) --more(2393)
        |--pstree(2392)
    |--multipathd(541) --{multipathd}(542)
        |--{multipathd}(543)
        |--{multipathd}(544)
        |--{multipathd}(545)
        |--{multipathd}(546)
        |--{multipathd}(547)
    |--networkd-dispat(635)
    |--polkitd(708) --{polkitd}(709)
        |--{polkitd}(711)
    |--rsyslogd(637) --{rsyslogd}(677)
        |--{rsyslogd}(678)
        |--{rsyslogd}(679)
    |--snapd(640) --{snapd}(683)
        |--{snapd}(684)
        |--{snapd}(686)
        |--{snapd}(687)
        |--{snapd}(688)
        |--{snapd}(713)
        |--{snapd}(714)
        |--{snapd}(791)
    |--systemd(2277) --- (sd-pam)(2278)
    |--systemd(2153) --- (sd-pam)(2154)
    |--systemd-journal(343)
    |--systemd-logind(641)
    |--systemd-network(392)
    |--systemd-resolve(592)
    |--systemd-timesyn(593) ---{systemd-timesyn}(597)
--More--
```


PROCESSOS

- **Foreground :**

- Processos específicos, executados por usuários, que estão visivelmente sendo processados.

- Ex:** execução de um comando ou duplo clique do mouse pra abrir um programa.

- **Background :**

- Processos com funções específicas criados pelo sistema operacional, que não dependem dos usuários para serem executados (*daemons*).

- Ex:** serviço de impressão, serviço de hospedagem web, entre outros.

CPU-BOUND E I/O-BOUND

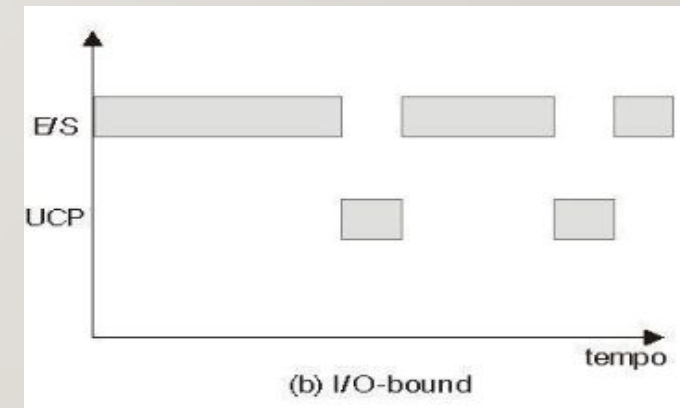
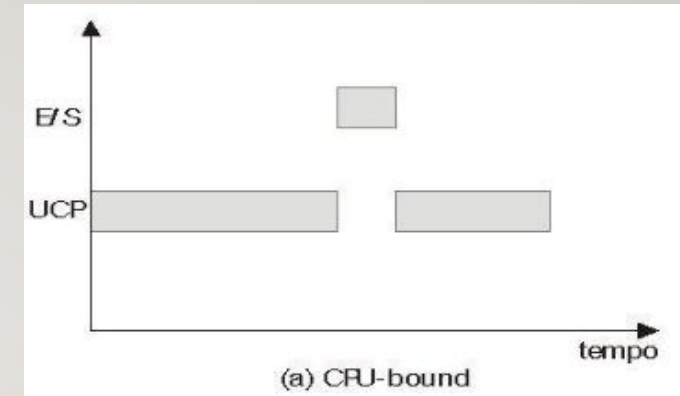
- **Processos *CPU-bound* :**

- Orientados à CPU.
- Tempo de execução definido pelo processador.

- **Processos *I/O-bound* :**

- Orientados à E/S.
- Tempo de execução definido pela duração das operações de E/S.

- **IDEAL:** balanceamento entre os dois tipos de processos.
- É o que os Sistemas de Propósito Geral costumam fazer.

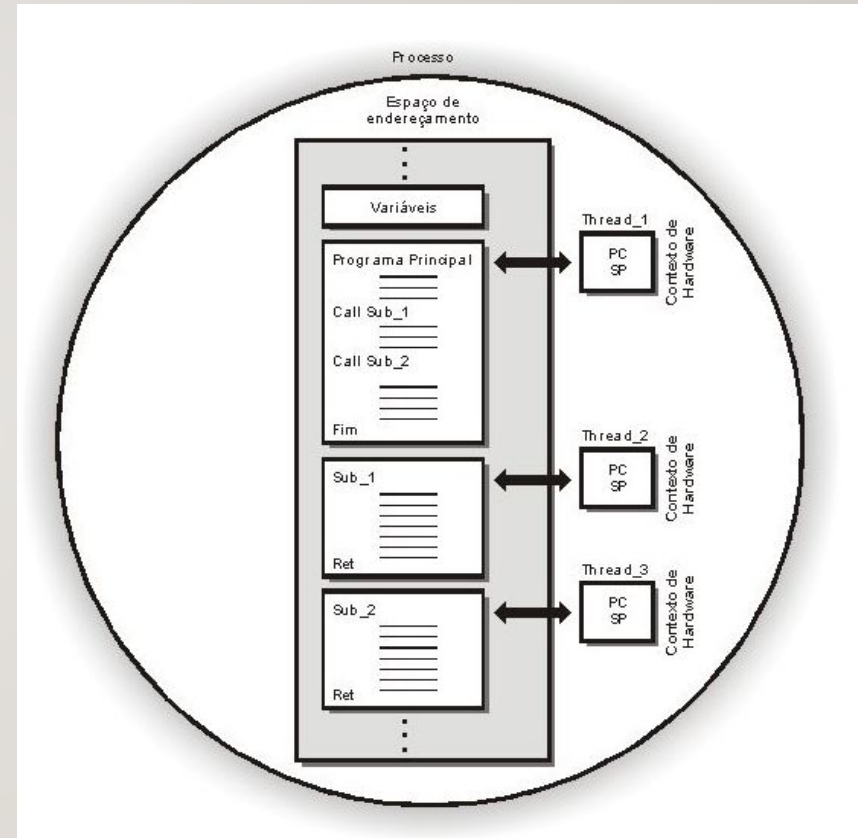


Processos CPU-bound e I/O-bound

Fonte: Arquitetura de Sistemas Operacionais, 4ª Edição, 2007

THREAD

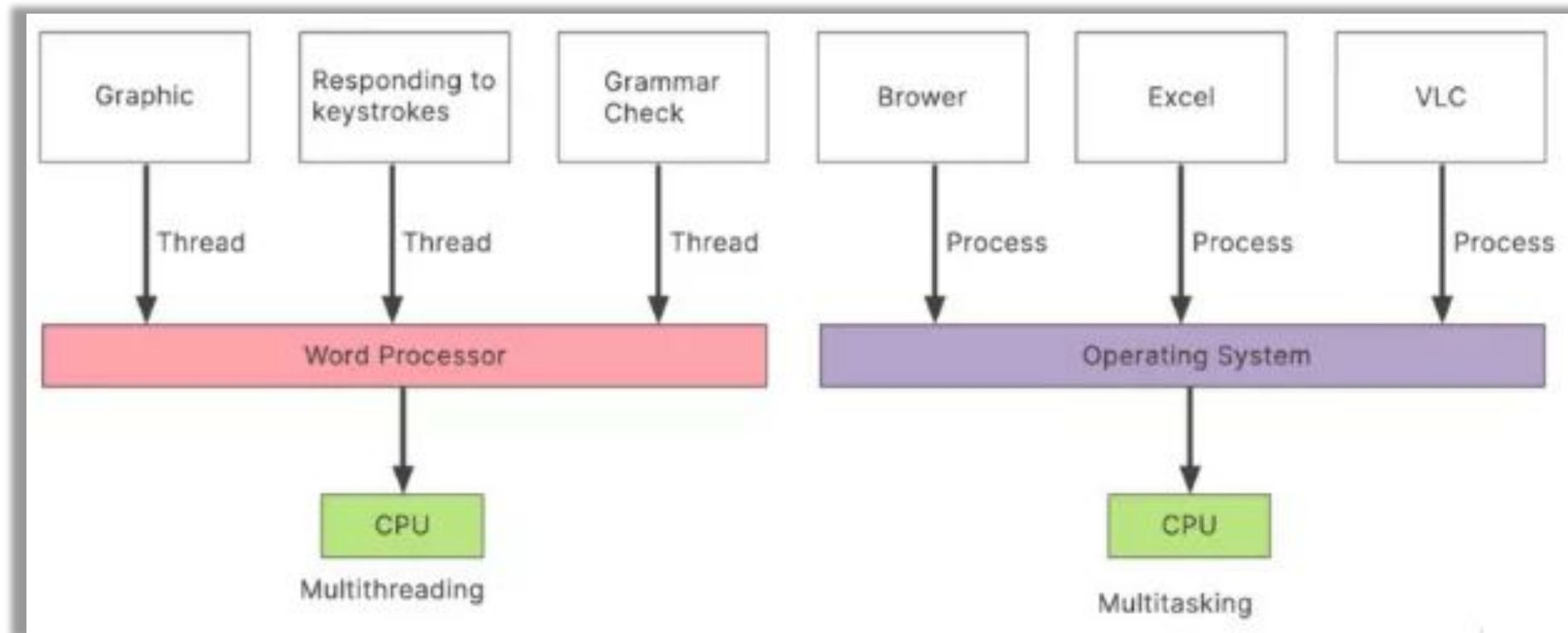
- **Thread:** unidade independente de um programa (pedaço) que se origina e executa em virtude de um processo.
- *Threads* compartilham um mesmo contexto de software e espaço de endereçamento !
- Existem threads de usuário e threads de kernel.
- **Sistemas Monothread:**
 - Só um programa faz parte do contexto de um processo.
- **Sistemas Multithread:**
 - Mais de um programa faz parte do contexto de um processo.



Aplicação Multithread

Fonte: Arquitetura de Sistemas Operacionais, 4ª Edição, 2007

MULTITHREADING X MULTITASK



Multithreading in SO

Fonte: Adaptado de <https://unstop.com/blog/multithreading-in-os>

EXERCÍCIO EM AULA

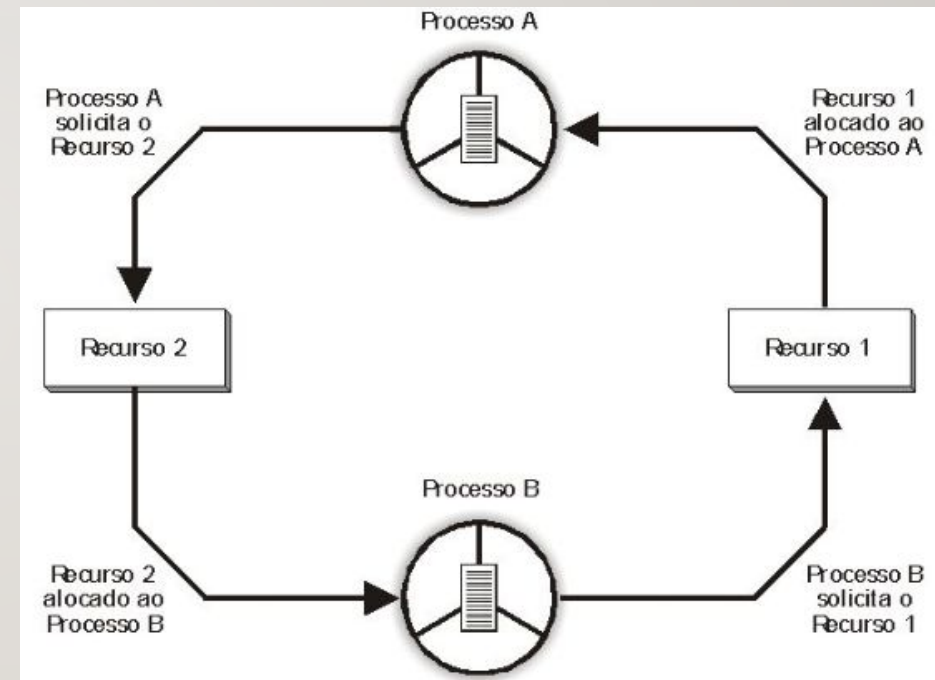
Abra o “Gerenciador de Tarefas” e vá na guia “Processos”. Observe esta guia e a deixe aberta por enquanto.

Abra seu navegador preferido, adicione várias abas e navegue em cada uma delas para um site diferente.

1. O navegador escolhido aparece no “Gerenciador de Tarefas” ?
2. Se sim, expanda seu processo. O que você vê?
3. Acompanhe o “Gerenciador de Tarefas” em tempo real. Agora, feche algumas das abas que abriu e olhe novamente no gerenciador. O que mudou? Por quê?

GESTÃO E DEADLOCK

- Lidar com inúmeros processos e *threads* não é tarefa simples.
- Exige sincronização na comunicação e compartilhamento de recursos para evitar inconsistências e *deadlocks*.
- **Espera Circular(Deadlock):**
 - ❑ Quando um processo aguarda um recurso que nunca estará disponível ou espera um evento que nunca ocorrerá.
- Soluções:
 - ❑ Só alocar recurso a um único processo por vez.
 - ❑ Recurso não pode ser liberado de um processo só porque outro o solicita.



Espera circular (Deadlock)

Fonte: Arquitetura de Sistemas Operacionais, 4ª Edição, 2007

ESCALONADOR

- Nível mais baixo do SO.
- Processo responsável pela manipulação de interrupções e execução de outros processos (mudanças de contexto).
- Acessa uma Tabela de Processos a serem atribuídos à CPU:
 - Cada entrada possui um ponteiro para o respectivo BCP (Bloco de Controle do Processo).
 - O BCP possui informações de contexto de hardware, de software e endereços de memória.

ponteiros
Estado do processo
Nome do processo
Prioridade do processo
Registradores
Limites de memória
Lista de arquivos abertos
⋮

Bloco de Controle do Processo (BCP)

Fonte: Arquitetura de Sistemas Operacionais, 4ª Edição, 2007

ESCALONAMENTO

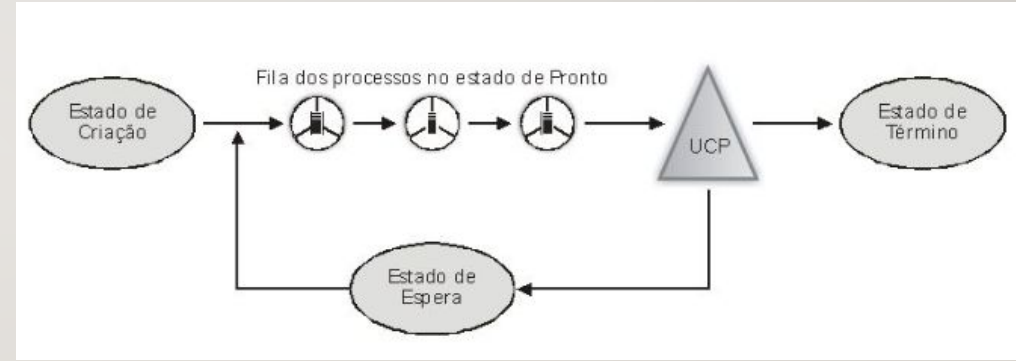
- Escolha do processo que ganhará vez na CPU. Realizado com o auxílio do hardware (*clocks*).
- 2,4 GHz $\square$ 2,4 bilhões de ciclos por segundo.
- O Escalonador deve se preocupar com a eficiência da CPU, pois os chaveamentos entre processos são complexos e custosos: afeta o sistema e a satisfação do usuário.
- **Algoritmo Não-preemptivo:**
 - $\square$ Processo é escolhido (escalonado) e executado até o fim.
- **Algoritmo Preemptivo:**
 - $\square$ Processo é executado até ser bloqueado por alguma razão (tempo, evento de E/S ou libere voluntariamente a CPU).

ESCALONAMENTO

- Independente do sistema, o ideal é que haja:
 - ❑ **Justiça:** cada processo deve receber uma parcela justa de tempo de CPU.
 - ❑ **Balanceamento e eficiência:** diminuir o máximo possível da ociosidade do sistema e recursos.
 - ❑ **Políticas do sistema:** prioridades entre processos.
- Veremos a seguir alguns exemplos comuns de Algoritmos de Escalonamento:
 - ❑ *First-In-First-Out (FIFO)*
 - ❑ *Shortest Job First (SJF)*
 - ❑ *Circular (Round-Robin)*
 - ❑ Prioridade
 - ❑ Circular com Prioridade

FIRST-IN-FIRST-OUT (FIFO)

- Também chamado de *First-Come, First Served (FCFS)*
- Implementação de fila simples.
- Primeiro que chegar ao estado de pronto é escalonado para execução.
- Não-preemptivo.
- Desvantagem:
 - Ineficiente quando há um processo que demora muito a ser executado.



Escalonamento FIFO

Fonte: *Arquitetura de Sistemas Operacionais*, 4ª Edição, 2007



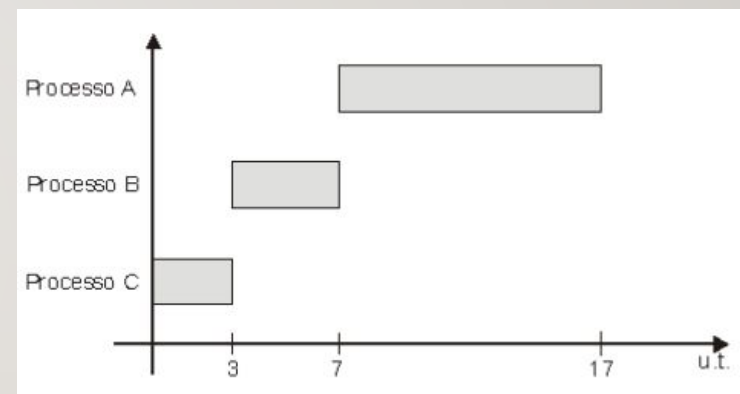
Vamos pensar um pouco ?

- I. Tomando como base a tabela a seguir, represente como ficaria o escalonamento e execução dos processos com Algoritmo FIFO.
 - a) Qual o Tempo de Espera de cada processo?
 - b) Qual o Tempo de Execução Total (*Turnaround*) de cada um?

Processo	Tempo para Processar (u.t.)
A	10
B	4
C	3

SHORTEST-JOB-FIRST (SJF)

- Menor processo é executado primeiro.
- Possível prever o tempo de execução do processo.
- Não-preemptivo.
- Desvantagem:
 - A medida que processos curtos surgem, eles tomam vez e os grandes demoram a ser executados.



Escalonamento SJF

Fonte: Arquitetura de Sistemas Operacionais, 4ª Edição, 2007



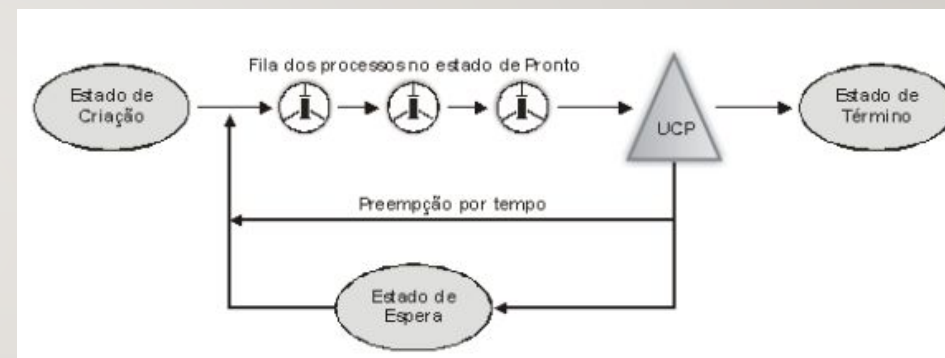
Vamos pensar um pouco ?

- I. Tomando como base a tabela a seguir, represente como ficaria o escalonamento e execução dos processos com Algoritmo SJF.
 - a) Qual o Tempo de Espera de cada processo?
 - b) Qual o Tempo de Execução Total (*Turnaround*) de cada um?

Processo	Tempo para Processar (u.t.)
A	11
B	7
C	3

CIRCULAR

- Também chamado de *Round-Robin*.
- Antigo, simples e ainda utilizado.
- Time-slice (*quantum*): tempo limite de uso da CPU.
- Preemptivo.
- Projetado para sistemas de tempo compartilhado.
- Desvantagem:
 - ❑ *Quantum* precisa estar muito bem ajustado.
 - ❑ Se pequeno, há muitas trocas: ineficiência.
 - ❑ Se muito longo: tempo de resposta comprometido.



Escalonamento Circular (Round-Robin)

Fonte: *Arquitetura de Sistemas Operacionais*, 4ª Edição, 2007



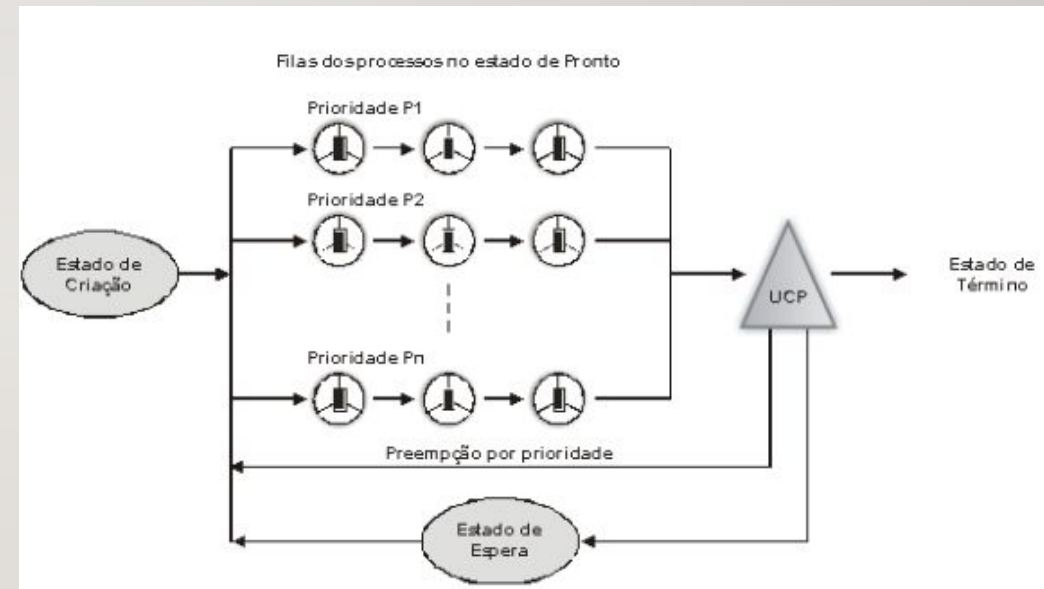
Vamos pensar um pouco ?

- I. Tomando como base a tabela a seguir, represente como ficaria o escalonamento e execução dos processos com Algoritmo Circular (*Round-Robin*) com um *quantum* de 3u.t. .
 - a) Qual o Tempo de Espera de cada processo?
 - b) Qual o Tempo de Execução Total (*Turnaround*) de cada um?

Processo	Tempo para Processar (u.t.)
A	10
B	4
C	3

PRIORIDADE

- Cada processo possui uma prioridade.
- Os de prioridade maior são executados primeiro.
- Atribuição de prioridades estáticas e dinâmicas.
- Não-preemptivo.
- Desvantagem:
 - Sem um chaveamento de prioridade entre os processos, as filas com menor prioridade quase nunca seriam executadas.



Escalonamento por Prioridade

Fonte: Arquitetura de Sistemas Operacionais, 4ª Edição, 2007



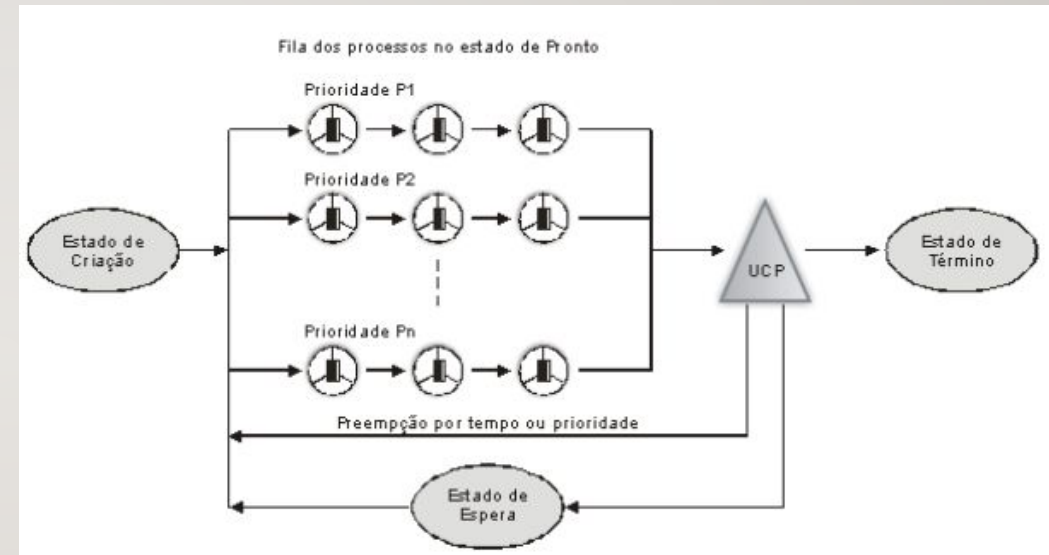
Vamos pensar um pouco ?

- I. Tomando como base a tabela a seguir, represente como ficaria o escalonamento e execução dos processos com Algoritmo por Prioridade.
 - a) Qual o Tempo de Espera de cada processo?
 - b) Qual o Tempo de Execução Total (*Turnaround*) de cada um?

Processo	Tempo para Processar (u.t.)	Prioridade
A	10	2
B	4	1
C	3	3

CIRCULAR COM PRIORIDADE

- Processos possuem prioridade, mas também executam de acordo com um *time-slice*.
- Preemptivo.
- Equilibra o uso da CPU em sistemas de tempo compartilhado.
- Amplamente utilizado em Sistemas de Propósito Geral das famílias Windows e Linux.



Escalonamento Circular com Prioridade

Fonte: *Arquitetura de Sistemas Operacionais*, 4ª Edição, 2007



Vamos pensar um pouco ?

- I. Tomando como base a tabela a seguir, represente como ficaria o escalonamento e execução dos processos com Algoritmo Circular por Prioridade com um *quantum* de 3u.t. .
 - a) Qual o Tempo de Espera de cada processo?
 - b) Qual o Tempo de Execução Total (*Turnaround*) de cada um?

Processo	Tempo para Processar (u.t.)	Prioridade
A	10	2
B	4	1
C	3	3



ALGUMA DÚVIDA?

NA PRÁTICA COM SOSIM



Fonte: Google Imagens

O QUE VIMOS HOJE?

- Conceitos sobre o que são processos, mudanças e seus estados
- Processos *Foreground* e *Background*, *CPU-bound* e *I/O-bound*
- Conceitos sobre *Threads*, *Monothreads* e *Multithreads*
- Escalonador e Algoritmos de Escalonamento
- Explorar o Simulador SOsim



Fonte: Google Imagens

PARA SABER MAIS...

- MAIA, L. P.. **SOsim: Simulador para o Ensino de Sistemas Operacionais Versão 2.0**. Disponível em: <<http://www.training.com.br/sosim/>>.
- TEDESCO, K.. **Concorrência, paralelismo, processos, threads, programação síncrona e assíncrona**. TREINAWEB. Disponível em: <<https://www.treinaweb.com.br/blog/concorrencia-paralelismo-processos-threads-programacao-sincrona-e-assincrona>>.
- THAKUR, S.. **What is Multithreading In OS? Understanding The Details**. Unstop Blog. Disponível em : <<https://unstop.com/blog/multithreading-in-os>>.

FIM
